

Gestão Financeira PF+PJ Implementação

Repositório, CI/CD, arquitetura de código e plano de construção fase a fase — escrito para uma IA executar e você validar.

Monorepo

Laravel 13 API

React 19

Expo

GitHub Actions

VERSÃO

1.0

DATA

**21 de agosto de
2026**

AUTOR

Deyvid + IA

STATUS

Para execução

Sumário

01	Contexto e premissas	4
02	Escopo técnico — dentro e fora	5
03	Stack confirmada	6
04	Infraestrutura e repositório	8
05	Arquitetura da aplicação	10
06	Componente central — captura plugável	12
07	Banco de dados	14
08	CI/CD — as três esteiras	16
09	Plano de construção — fases	18
10	Decisões técnicas	21
11	Riscos técnicos	22

Contexto e premissas

Este documento assume o produto já decidido — o **o quê** está no documento de concepção (v1.3). Aqui é o **como**: repositório, pipeline, arquitetura de código, e a ordem em que uma IA deve construir isto sem quebrar o que já funciona.

Volume esperado: **um usuário, N empresas** (hoje uma, projeta crescer). Isso não é um detalhe — é o que autoriza várias simplificações que um SaaS multiusuário não teria: sem tenant isolation complexo, sem rate limiting agressivo, sem fila de alta concorrência. O gargalo nunca vai ser performance; vai ser o tempo que você tem para revisar o que a IA construiu.

PREMISSA QUE MUDA O PLANO INTEIRO

Quem desenvolve é a IA; quem testa e aprova é você. Isso significa que o critério de qualidade de cada bloco não é "compila e funciona" — é "pequeno o bastante para você revisar em uma sessão, com um PR que explica o que mudou e como testar". Um bloco grande demais quebra esse ciclo: você para de conseguir validar de verdade e passa a aprovar por cansaço.

1.1 O que já está decidido e não é reaberto aqui

Todas as decisões D-01 a D-14 do documento de concepção são premissa. As mais relevantes para este documento: Laravel 13/PHP 8.3/MySQL 8 (D-01, D-02), multiempresa desde o início (D-03), Pluggy como agregador mas **desligado por padrão** (D-05), sem consulta automática de boleto ao BC — e-mail e Telegram como portas prioritárias (D-06, D-08), app via Expo consumindo API versionada (D-09), MFA + biometria desde o início (D-10), uso pessoal sem intenção de produtizar (D-11).

1.2 O que muda de novo neste documento

Três coisas que o documento de concepção não detalhava e que este resolve: como o código se organiza em repositórios e branches, como o deploy acontece sem servidor próprio (três esteiras de CI/CD para HostGator + EAS), e como os canais de captura (e-mail, Telegram, e futuramente Pluggy) entram no sistema por contrato — nunca por chamada direta — para que ligar um agregador de verdade na F2 não signifique reabrir código que já está em produção desde a F0.

Escopo técnico — dentro e fora

A lista "fora" é a mais importante desta seção — é o que impede a IA de "ajudar" implementando algo que ainda não deveria existir.

2.1 Dentro — o que se constrói já, sem esperar terceiro

- Toda a fundação (F0): auth, contas, cartões, boletos, lançamentos, investimentos, categorias — **100% manual, zero API externa**.
- Simulador, fluxo de caixa, metas, motor de obrigações recorrentes (F1) — rodam sobre dado que já está no banco.
- Motor de e-mail e bot do Telegram (F1) — **já entram nesta rodada**, preparados via interface, porque são gratuitos, sem OAuth, e não têm data de "quando estiver pronto para produzir" — funcionam desde o primeiro dia de uso pessoal.

2.2 Fora — adiado de propósito

- **Pluggy/Open Finance (F2)**: interface pronta desde a F0 (`BankAggregatorInterface`), implementação real só quando você ligar — não é ausência de código, é código desligado por config.
- **App Expo (F3)**: depende de F0/F1 estarem validados no uso real primeiro.
- **Integra Contador/e-CNPJ (D-07 em aberto)**: sem certificado, sem chamada à Receita — motor de obrigações manual cobre o essencial por ora.
- **Rentabilidade automática de investimentos (D-14)**: fora por decisão, não por limitação técnica.

ANTIPADRÃO A EVITAR

Não adiante a implementação de Pluggy "já que a interface está pronta mesmo". A interface existir não é convite para implementá-la cedo — é o contrário: ela existe justamente para que a implementação real possa esperar sem custo de retrabalho. Adiantar código de F2 dentro da F0 é o tipo de iniciativa que parece produtiva e na prática atrasa o que faz diferença primeiro (F0/F1 funcionando no seu dia a dia).

Stack confirmada

Versões verificadas por busca em agosto/2026 — não presumidas.

PEÇA	VERSÃO	POR QUÊ
Laravel	13 (lançado mar/2026)	Já decidido (D-01); PHP 8.3 mínimo, zero breaking change anunciado a partir do Laravel 12
PHP	8.3+	Mínimo exigido pelo Laravel 13; usar a mais alta disponível no HostGator
MySQL	8	JSON, FULLTEXT, CTEs — já decidido (D-02)
React	19.2.x	Versão estável atual; sem motivo para fixar numa anterior
Vite	7.x	Build 100% estático — sem Node rodando no servidor
Expo SDK	55 (React Native 0.83)	Confirmar a mais recente no momento real da F3 — SDK do Expo evolui rápido
Sanctum	pacote oficial Laravel	Token de API para web e mobile — já decidido no doc. de concepção
Pint	PSR-12	Padrão DMTA — roda antes de todo commit
Pest	testes	Padrão DMTA

SOBRE TRAVAR VERSÃO EXATA DO EXPO

Este documento não fixa um SDK do Expo como definitivo porque a F3 só começa depois de F0/F1/F2 — e o SDK do Expo tem ciclo de release rápido. A regra é: confirme a versão estável atual por busca no início da F3, não use o número escrito aqui sem checar.

Infraestrutura e repositório

Um monorepo, três apps, três esteiras de deploy independentes.

4.1 Por que monorepo, não três repositórios

A alternativa óbvia — um repositório por app — existe para isolar times com ciclos de release diferentes. Aqui não há esse problema: existe um único desenvolvedor (a IA) e um único revisor (você). O custo real de três repositórios não é o setup inicial, é a manutenção contínua: três READMEs, três configs de CI, e o risco concreto de o contrato de API divergir sem ninguém perceber até o deploy. Num monorepo, a IA vê os três lados na mesma sessão de trabalho e mantém o contrato coerente.

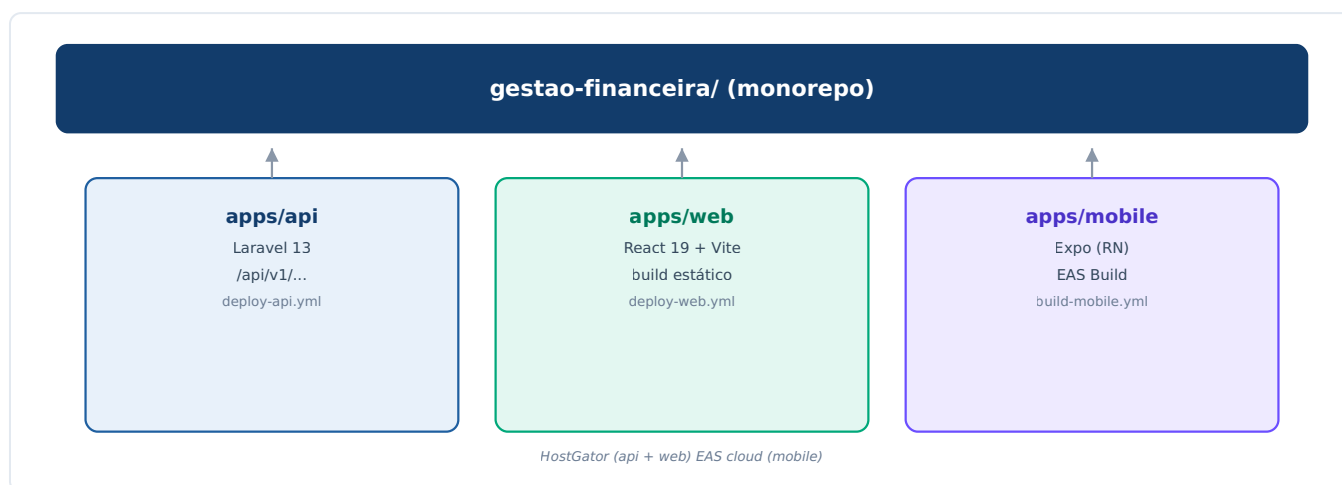


Figura 1 — um repositório, três apps, três esteiras de deploy independentes por caminho alterado.

4.2 Ponto de partida do apps/api

Clonar `laravel-base` (github.com/deyvidspindola/laravel-base) para dentro de `apps/api`, remover o histórico git próprio do repositório clonado, e seguir a partir dali — já traz Docker local e o padrão de código DMTA aplicado, evitando reescrever a base do zero.

PENDÊNCIA — DT ABERTA

O conteúdo real do `laravel-base` ainda não foi inspecionado — esta seção usa como referência a skill de padrões (`padroes-laravel-dmta`), que documenta a convenção, não necessariamente o código exato do repositório. Assim que o caminho local for compartilhado, esta seção e o plano de construção da F0 devem ser revisados contra o repositório real.

4.3 Ambiente local

`docker-compose.yml` replicando as restrições do HostGator (sem Redis, sem worker permanente, sem WebSocket) — `nginx/php-fpm`, `mysql`, `node` só para Vite em modo dev, um contêiner `scheduler` rodando `schedule:run` em loop, e `mailpit` capturando todo e-mail local. Regra central da skill de padrões: **o ambiente local não pode ter nada que a produção não tenha** — é o que faz um problema de memória ou de fila aparecer no seu notebook, não só depois do deploy.

Arquitetura da aplicação

Mesma Clean Architecture do padrão DMTA — UseCases/Services/Domain/Models — com um acréscimo: a camada de captura plugável do capítulo 06.

5.1 Camadas (apps/api)

CAMADA	ONDE	CONTÉM
Controller	app/Http/Controllers/ Api/V1/	Localizar recurso, autorizar (Sanctum + Policy), delegar, formatar resposta
UseCase	app/UseCases/	Uma intenção por classe: <code>RegisterTransaction</code> , <code>SimulateInstallmentPurchase</code>
Service	app/Services/	Lógica reusada por 2+ casos de uso, sem estado
Domain	app/Domain/	Regra pura: calculadora de CET, avaliador de semáforo de comprometimento, interfaces de captura (cap. 06)
Model	app/Models/	Eloquent — fillable, casts, relations, scopes. Nada de regra de negócio

Estrutura de pastas, limites de tamanho de arquivo, PHPDoc obrigatório e nomenclatura seguem a skill `padroes-laravel-dmta` sem exceção — não repetido aqui para não divergir de uma fonte só.

5.2 Versionamento de API — /api/v1

Toda rota nasce sob `/api/v1/...` desde a F0, mesmo antes de o app Expo existir (D-09). Trocar de versão depois que web e mobile já dependem de uma rota sem versionamento é o tipo de retrabalho evitável escrevendo uma palavra a mais no arquivo de rotas hoje.

5.3 apps/web e apps/mobile

Ambos consomem a mesma API via token Sanctum. `apps/web` é SPA React com build estático (sem SSR — não há Node no servidor de produção). `apps/mobile` só nasce na F3; construir o projeto Expo vazio antes disso é manter uma dependência para atualizar sem nenhum uso — adiado de propósito (ver capítulo 02).

Componente central — captura plugável

O coração técnico deste sistema não é uma tela — é o contrato que faz e-mail, Telegram, scanner e (depois) Pluggy alimentarem o mesmo caso de uso sem o caso de uso saber de onde veio o dado.

6.1 O contrato

```
interface TransactionCaptureChannelInterface {
    public function origin(): string; // CaptureOrigin::Manual|Email|Telegram|Scanner|Aggregator
}

interface EmailBoletoReaderInterface extends TransactionCaptureChannelInterface {
    public function readAttachment(string $pdfPath): BoletoDraftData;
}

interface QuickEntryChannelInterface extends TransactionCaptureChannelInterface {
    public function parseMessage(string $chatId, string $message): ?TransactionDraftData;
}

interface BankAggregatorInterface {
    public function isEnabled(): bool;
    public function sync(AggregatorConnection $connection): SyncResult;
}
```

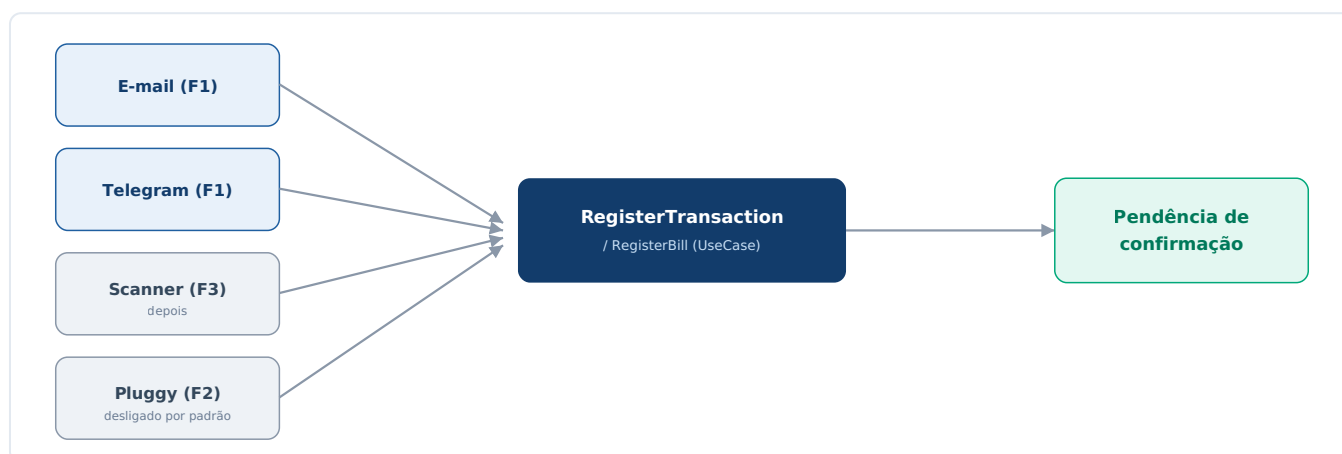


Figura 2 — todo canal converge para o mesmo caso de uso; a origem vira dado, não código diferente.

6.2 O que isso destrava

Ligar Pluggy na F2 vira uma linha de config (`AGGREGATOR_ENABLED=true`) e uma implementação nova de `BankAggregatorInterface` — não uma reescrita de `RegisterTransaction` , que já está em produção desde a F0/F1 processando lançamentos manuais, de e-mail e de Telegram.

Banco de dados

Herda o modelo do capítulo 12 do documento de concepção; aqui só o que muda para sustentar a captura plugável.

TABELA/COLUNA	OBSERVAÇÃO TÉCNICA
<code>categories.parent_id</code>	Autorreferente, nullable — subcategoria aponta para a categoria-mãe (D-12)
<code>*.origin</code>	Enum <code>CaptureOrigin</code> em toda tabela que recebe lançamento por canal externo (<code>bills</code> , <code>statement_entries</code>)
<code>*.status</code>	<code>pending_confirmation</code> / <code>confirmed</code> — nunca confirma sozinho um dado vindo de e-mail/Telegram/agregador
<code>aggregator_connections</code>	Existe desde a F0 (schema), populada só a partir da F2
<code>company_id</code> / <code>context_id</code>	Presente em toda tabela de negócio — nunca uma query sem esse filtro

ANTIPADRÃO A EVITAR

Criar a tabela `aggregator_connections` só na F2 "porque ainda não precisa" parece disciplina, mas gera uma migration tardia que mexe em schema já usado em produção. O schema nasce na F0 (parte do modelo de dados completo), só o dado dentro dele fica vazio até a F2 — schema e uso têm calendários diferentes de propósito.

CI/CD — as três esteiras

Cada workflow dispara só quando sua própria pasta muda — nada de rodar deploy da API por causa de um commit no app mobile.

8.1 deploy-api.yml

Gatilho: push em `main` tocando `apps/api/**`. Passos: checkout → PHP 8.3 → `composer install --no-dev` → `pint --test` → `php artisan test` → deploy via SSH (`easingthemes/ssh-deploy`, `rsync`) → `migrate --force` + `config:cache` + `route:cache` rodados remotamente por SSH no mesmo workflow (`appleboy/ssh-action`). Document root fica fora de `public_html` — só `apps/api/public` é exposto, regra não negociável do padrão DMTA.

8.2 deploy-web.yml

Gatilho: push em `main` tocando `apps/web/**`. Checkout → Node 22 → `npm ci` → `npm run build` (Vite gera build estático) → deploy do conteúdo de `dist/` via FTP (ou SSH/`rsync`, sem diferença técnica aqui — não há comando remoto a rodar depois de um arquivo estático) para uma subpasta pública. Nenhum processo Node roda no servidor.

8.3 build-mobile.yml

Gatilho **manual** (`workflow_dispatch`), não automático — build mobile consome cota do EAS e você decide quando gerar um novo. Usa `expo/expo-github-action@v8` e `eas build --platform android --profile preview --non-interactive`.

SECRET	USADO POR
<code>SSH_HOST</code> , <code>SSH_USER</code> , <code>SSH_PORT</code> , <code>SSH_PRIVATE_KEY</code>	deploy-api — chave dedicada ao deploy, não a pessoal
<code>FTP_SERVER</code> , <code>FTP_USERNAME</code> , <code>FTP_PASSWORD</code>	deploy-web (enquanto ficar em FTP)
<code>EXPO_TOKEN</code>	build-mobile

DECIDIDA — DT-06

Plano HostGator confirmado com SSH. Deploy da API sai do FTP e vai por SSH/`rsync`, com os comandos `artisan` pós-deploy rodando no mesmo workflow — sem rota HTTP temporária de migração, sem passo manual.

Plano de construção — fases

Mesma numeração F0-F3 do documento de concepção. O detalhe passo a passo de cada uma vive em `docs/fases/*.md` — aqui, a visão executiva.

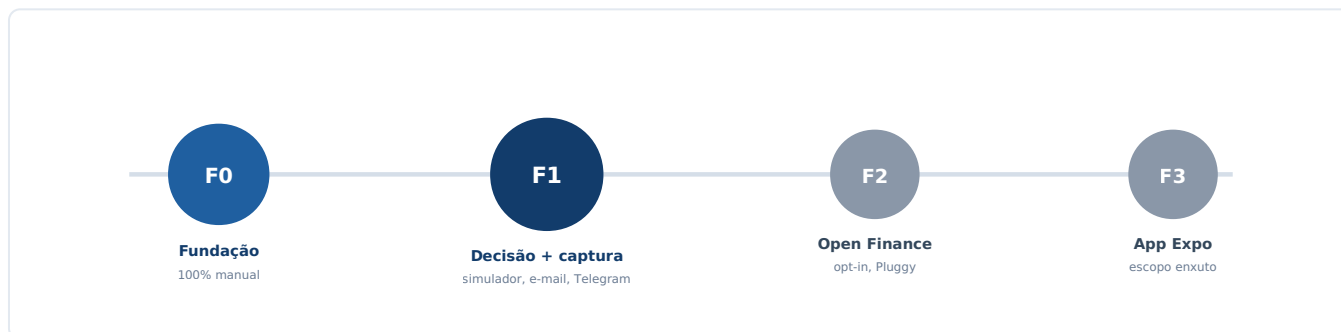


Figura 3 — F0 e F1 não dependem de nenhum terceiro; F2 é opt-in; F3 fecha o ciclo.

9.1 F0 — Fundação

Monorepo, Docker local, Sanctum + MFA, modelo de dados completo, casos de uso de cadastro manual, dashboards por contexto. **Pronto quando:** você cadastra conta/cartão/categoria/boleto/lançamento para PF e uma empresa, vê os três dashboards corretos, e os dois primeiros workflows de deploy publicam com sucesso.

9.2 F1 — Decisão e captura

Simulador (CET, cenários, mês mais apertado), metas, motor de obrigações recorrentes, motor de e-mail, bot do Telegram. **Pronto quando:** um e-mail encaminhado com boleto e uma mensagem no Telegram viram pendência de confirmação, e uma simulação real de parcelamento mostra impacto e gráfico.

9.3 F2 — Open Finance

Opt-in. Implementação real de `PluggyAggregator`, fluxo de consentimento, sincronização agendada. **Pronto quando:** uma conta real sincroniza extrato automaticamente sem se misturar com lançamento manual.

9.4 F3 — App Expo

Escopo enxuto: login, dashboard de leitura, lançamento rápido, scanner de código de barras, confirmação de pendências. **Pronto quando:** você usa o app no Android para confirmar um lançamento do Telegram sem abrir o navegador.

Decisões técnicas

Seis decididas, zero em aberto. Detalhe completo em `00_DECISOES_TECNICAS.md`.

ID	STATUS	DECISÃO
DT-01	Decidida	Monorepo único, não três repositórios
DT-02	Decidida	Branch curta por tarefa + PR obrigatório + Conventional Commits em inglês
DT-03	Decidida	Interfaces plugáveis para e-mail, Telegram e agregador bancário
DT-04	Decidida	Três GitHub Actions por path, build mobile com gatilho manual
DT-05	Decidida	Agregador Pluggy desligado por padrão (<code>AGGREGATOR_ENABLED=false</code>)
DT-06	Decidida	Deploy da API por SSH/rsync (plano HostGator confirmado com SSH), não FTP

Riscos técnicos

O que pode dar errado especificamente na execução — não repete os riscos de produto já listados no documento de concepção.

RISCO	GRAVIDADE	MITIGAÇÃO
PR grande demais para revisar de verdade	Alta	Uma tarefa por branch, escopo definido pelos itens do próprio arquivo de fase — nunca "implementar a F1 inteira" numa branch só
Chave SSH de deploy com permissão ampla demais	Média	Chave dedicada só ao deploy, restrita ao diretório do projeto no servidor — nunca reusar a chave SSH pessoal
Credencial de SSH/Telegram/Expo vazando no repositório	Alta	Tudo via GitHub Secrets, nunca em <code>.env</code> commitado — checklist de commit já cobre isso (padrão DMTA)
Implementação de Pluggy adiantada "por estar fácil"	Média	Antipadrão nomeado no capítulo 02 — a interface pronta não é convite para implementar cedo
Fila do EAS Build (plano gratuito) atrasar builds	Baixa	Build manual, não por commit — você decide quando vale esperar

PRÓXIMOS PASSOS IMEDIATOS

1. Gerar uma chave SSH dedicada ao deploy (não a pessoal) e cadastrar nos secrets do GitHub.
2. Compartilhar o caminho local do `laravel-base` para revisão da seção 4.2 e da F0.
3. Criar o monorepo e a primeira branch: `feature/f0-setup-monorepo`.
4. Confirmar o caminho absoluto do projeto no servidor para preencher o workflow.